AUDIT REPORT

Burnt Labs

Treasury Contract

Prepared by SCV-Security

On 30th July 2025

Table of Contents

Table of Contents **2**
Introduction **3**
Scope Functionality 3
Submitted Codebase 4
Revisions – Submitted Codebase 4
Methodologies 4
Code Criteria 5
Findings Summary **6**
Findings Technical Details **7**
1. Attackers can waste fee funds to cause a denial of service 7
2. Admin address is not validated 9
3. Potential empty removal of state entries 10
4. Two-step ownership transfer is not implemented 11
Document Control **12**
Appendices **13**
A. Appendix - Risk assessment methodology 13
B. Appendix - Report Disclaimer 14

Introduction

SCV has been engaged by Burnt Labs to conduct a comprehensive security review with the goal of identifying potential security threats and vulnerabilities within the codebase. The purpose of this audit is to evaluate the security posture of the codebase and provide actionable recommendations to mitigate any identified risks. This report presents an overview of the findings from our security audit, outlining areas of concern and proposing effective measures to enhance the codebase's security.

Scope Functionality

The treasury contract allows dapps to pay transaction fees on behalf of users via `authz` grants. Before receiving a fee grant, users must satisfy the grant requirements configured by the contract admin. Some of the functionalities include allowing the contract admin to update the grant requirements, revoke any existing grants, and update the fee allowance amount.

Submitted Codebase

Treasury Contract	
Repository	https://github.com/burnt-labs/contracts
Commit	`762e130b1f5f9963781374f539aba764cc20ed7c`
Contract	`treasury`
Branch	`main`

Revisions – Submitted Codebase

Treasury Contract	
Repository	https://github.com/burnt-labs/contracts
Commit	`2b707c4bc0b05a0bd2b56b7a6bd8ad80060b1ba8`
Contract	`treasury`
Branch	`main`

Methodologies

SCV performs a combination of automated and manual security testing based on the scope of testing. The testing performed is based on the extensive experience and knowledge of the auditor to provide the greatest coverage and value to Burnt Labs. Testing includes, but is not limited to, the following:

- Understanding the application and its functionality purpose.
- Deploying SCV in-house tooling to automate dependency analysis and static code review.
- Analyse each line of the code base and inspect application security perimeter.
- Review underlying infrastructure technologies and supply chain security posture.

Code Criteria

This section provides an evaluation of specific criteria aspects as described below:

- **Documentation:** Evaluating the presence and comprehensiveness of publicly available or provided explanatory information, diagram flowcharts, comments, and supporting documents to enhance code understanding.
- **Coverage:** Evaluating whether the code adequately addresses all necessary cases and scenarios, ensuring that the intended functionality or requirements are sufficiently covered.
- **Readability:** Assessing how easily the code can be understood and maintained, considering factors such as code structure, naming conventions, and overall organisation.
- **Complexity:** Evaluating the complexity of the code, including factors such as, number of lines, conditional statements, and nested structures.

The status of each criteria is categorised as either **SUFFICIENT** or **NOT-SUFFICIENT** based on the audit assessment. This categorisation provides insights to identify areas that may require further attention and improvement.

Criteria	Status	Notes
Documentation	**NOT-SUFFICIENT**	While no documentation was provided, a kickoff meeting effectively provided the necessary context for this audit.
Coverage	**SUFFICIENT**	N/A
Readability	**SUFFICIENT**	N/A
Complexity	**SUFFICIENT**	N/A

Findings Summary

Summary Title	Risk Impact	Status
Attackers can waste fee funds to cause a denial of service	SEVERE	ACKNOWLEDGED
Admin address is not validated	LOW	RESOLVED
Potential empty removal of state entries	INFO	RESOLVED
Two-step ownership transfer is not implemented	INFO	RESOLVED

Findings Technical Details

1. Attackers can waste fee funds to cause a denial of service

RISK IMPACT: SEVERE	STATUS: ACKNOWLEDGED

Revision Notes

The contract's deploy_fee_grant function is intentionally designed to be permissionless, allowing any user to request a fee grant. While this supports open accessibility, it introduces a configuration risk that can lead to Denial-of-Service (DoS) through fee exhaustion.

The client acknowledges that this is not a vulnerability in the contract logic itself but rather a byproduct of how the contract is configured and used. The contract correctly enforces GRANT_CONFIGS constraints, and usage patterns such as periodic or temporal limits are available and encouraged. However, if these limits are misconfigured or used without sufficient scope restrictions (e.g., specific contract addresses or methods), attackers may exploit the open-ended configuration to drain fee funds.

Description

The `deploy_fee_grant` function grants a fee allowance for the grantee to use a specified amount of funds from the contract to pay transaction fees. The contract performs validation to ensure the `GRANT_CONFIGS` requirements are satisfied before issuing a fee grant.

The issue is that an attacker can maliciously waste fees in the contract to prevent legitimate users from using it with the following steps:

1. The attacker creates two accounts that are controlled by them.
2. The attacker uses the first account to grant the required `GRANT_CONFIGS` to the second account.

3. The attacker calls the `deploy_fee_grant` function with their two accounts as the `authz_granter` and `authz_grantee` parameters. This will cause their second account to receive a fee grant.
4. The attacker uses their second account to repeatedly dispatch transactions that will fail in order to consume the granted fees.
5. Once the granted fees are fully consumed, the attacker calls the `deploy_fee_grant` function again to receive a new grant. Fee grants that implement threshold limits (e.g., `PeriodicAllowance`) can be bypassed using the same method.
6. The attacker repeats steps 4 to 5 to consume all funds in the contract.

Recommendation

Consider adding authentication checks so only the admin can issue fee grants.

2. Admin address is not validated

RISK IMPACT: LOW	STATUS: RESOLVED

Description

When the `ADMIN` field is initialized and updated, the supplied `Addr` field is not validated to ensure the address inside is valid. This is required because the `Addr` field can be initialized with `Addr::unchecked` to include an incorrect address format.

Recommendation

Consider validating the admin address with the `addr_validate()` function in the `init` and `update_admin` entry points.

3. Potential empty removal of state entries

RISK IMPACT: INFORMATIONAL	STATUS: RESOLVED

Description

The `remove_auth_method` and `remove_grant_config` functions directly remove the `AUTHENTICATORS` and `GRANT_CONFIGS` keys from the state without validating that the key entry exists in storage. If the provided authenticator ID or `msg_type_url` does not exist, no error will be thrown, which will mislead the caller into thinking that the transaction performs an actual state mutation.

Recommendation

Consider adding a `has()` validation to ensure only keys that exist in the state are removed in the `remove_auth_method` and `remove_grant_config` functions.

4. Two-step ownership transfer is not implemented

RISK IMPACT: INFORMATIONAL	STATUS: RESOLVED

Description

The `treasury` contract does not implement a two-step ownership transfer when updating the `ADMIN` address. As a result, the ownership will be lost and cannot be recovered if transferred to an incorrect address that no one owns.

Recommendation

Consider implementing a two-step ownership transfer that proposes a new owner in the first step and requires the proposed owner to accept it in the second step. Using a two-step ownership transfer mechanism helps provide a window of opportunity for the current owner to cancel the transfer if they did not intend to initiate it or if there were any unintended actions.

Document Control

Version	Date	Notes
-	28th November 2024	Security audit commencement date.
0.1	7th December 2024	Initial report with identified findings delivered.
0.5	14th December 2024	Fixes remediations implemented and reviewed.
1.0	30th July 2025	Final report Release.

Appendices

A. Appendix – Risk assessment methodology

SCV-Security employs a risk assessment methodology to evaluate vulnerabilities and identified issues. This approach involves the analysis of both the LIKELIHOOD of a security incident occurring and the potential IMPACT if such an incident were to happen. For each vulnerability, SCV-Security calculates a risk level on a scale of 5 to 1, where 5 denotes the highest likelihood or impact. Consequently, an overall risk level is derived from combining these two factors, resulting in a value from 10 to 1, with 10 signifying the most elevated level of security risk

Risk Level	Range
CRITICAL	10
SEVERE	From 9 to 8
MODERATE	From 7 to 6
LOW	From 5 to 4
INFORMATIONAL	From 3 to 1

LIKELIHOOD and **IMPACT** would be individually assessed based on the below:

Rate	LIKELIHOOD	IMPACT
5	**Extremely Likely**	Could result in severe and irreparable consequences.
4	**Likely**	May lead to substantial impact or loss.
3	**Possible**	Could cause partial impact or loss on a wide scale.
2	**Unlikely**	Might cause temporary disruptions or losses.
1	**Rare**	Could have minimal or negligible impact.

B. Appendix – Report Disclaimer

This report should not be regarded as an "endorsement" or "disapproval" of any specific project or team. These reports do not indicate the economics or value of any "product" or "asset" created by a team or project that engages SCV-Security for a security review. The audit report does not make any statements or warranties about the code's utility, safety, suitability of the business model, regulatory compliance of the business model, or any other claims regarding the fitness of the implementation for its purpose or its bug-free status. The audit documentation is intended for discussion purposes only. The content of this audit report is provided "as is," without representations and warranties of any kind, and SCV-Security disclaims any liability for damages arising from or in connection with this audit report. Copyright of this report remains with SCV-Security.

THANK YOU FOR CHOOSING



SCV SECURITY

scv.services

contact@scv.services